AWS | Student Project Report

VPC Peering

Connect two VPCs. Prove private connectivity.

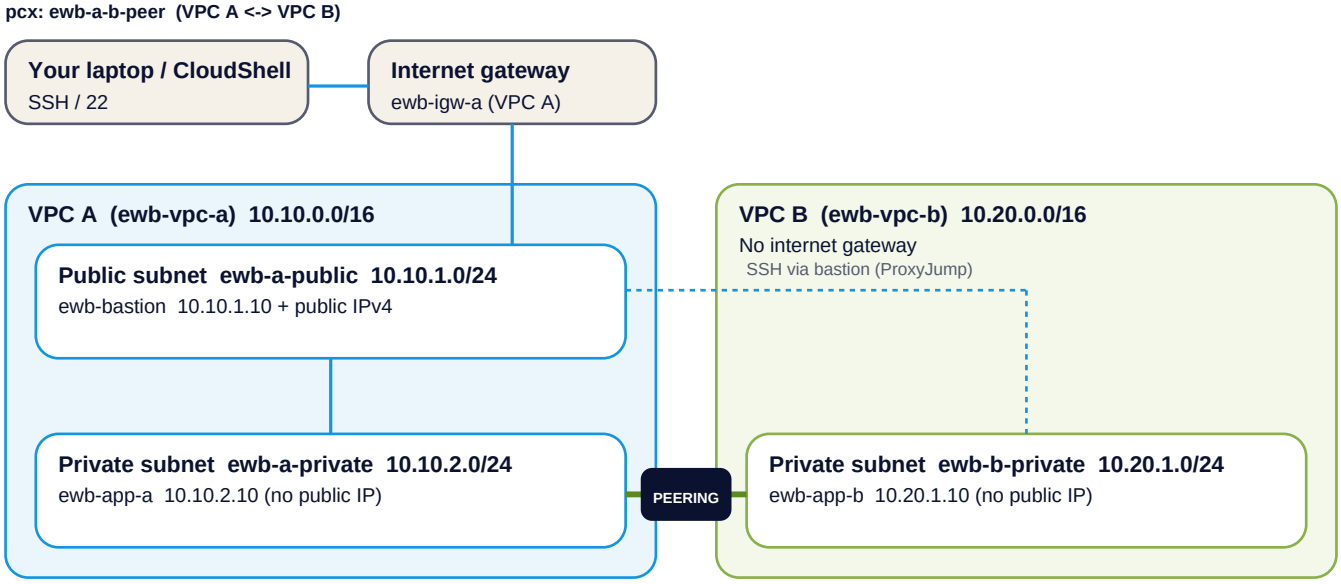
Name	Om Panwar
Batch	AWS Internship Batch 1
Project title	VPC Peering (Project 01)
Programme	AWS Cloud Computing Internship - EWB Edutech Private Limited
Region	Asia Pacific (Sydney) ap-southeast-2 (Availability Zone ap-southeast-2a)
Completion date	26 October 2026

Report contents

1. Architecture and resource inventory 2. Subnets, internet gateway and route tables 3. Peering connection and security group rules 4. Test evidence (baseline, forward, reverse, App-B SSH) 5. Controlled failure and recovery 6. Forward and return path explanation 7. Viva answers 8. What I learned 9. Cleanup evidence

1. Architecture and resource inventory

One internet entry point (bastion in VPC A), two private application servers, and a direct VPC peering connection between VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16). The two CIDRs do not overlap, which is required for peering.



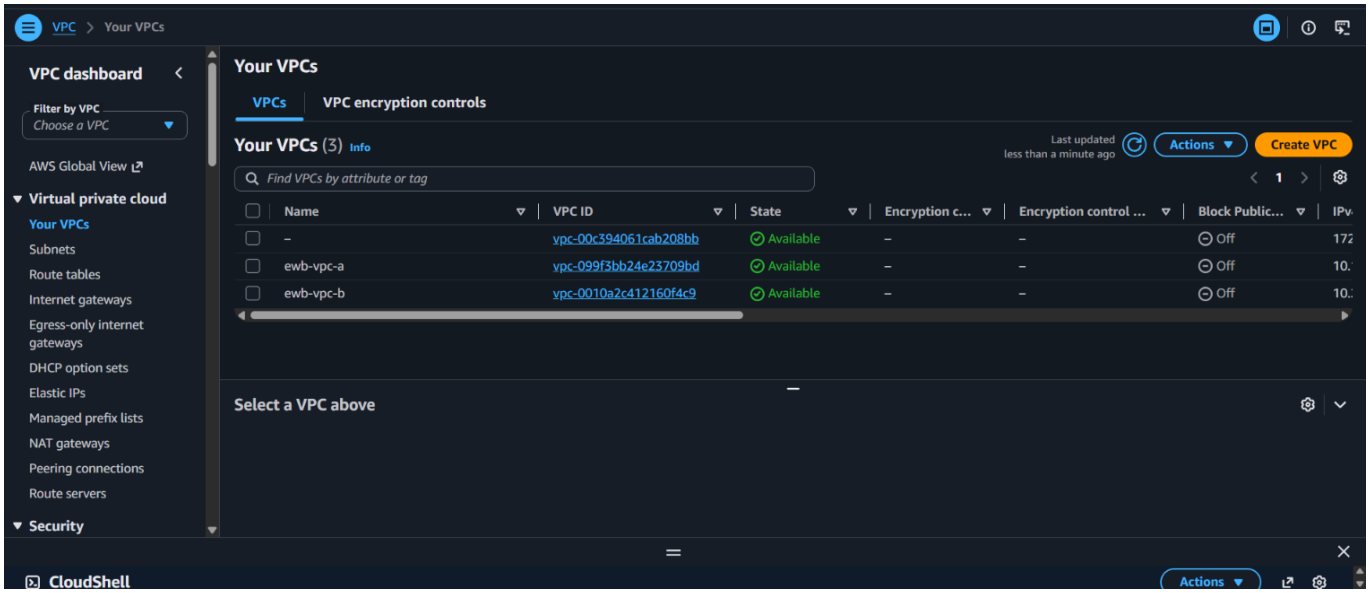
Resource inventory (actual IDs)

Resource / Name tag	ID	CIDR / address
ewb-vpc-a	vpc-099f3bb24e23709bd	10.10.0.0/16
ewb-vpc-b	vpc-0010a2c412160f4c9	10.20.0.0/16
ewb-a-public	subnet-091cacbd081ee996e	10.10.1.0/24
ewb-a-private	subnet-00fecaf6a58e296d7	10.10.2.0/24
ewb-b-private	subnet-0ee8c65e2b6e7ee3b	10.20.1.0/24
ewb-igw-a	igw-0f92fd2caf269234f	Attached to ewb-vpc-a
ewb-rt-a-public	rtb-0d71c6248f127bd67	Associated: ewb-a-public
ewb-rt-a-private	rtb-0adb7a56cd1ae99c6	Associated: ewb-a-private
ewb-rt-b-private	rtb-0a6355f693a1442e2	Associated: ewb-b-private
ewb-a-b-peer	pcx-0c2371ac171a2f4f0	Requester VPC A / Accepter VPC B - Active
ewb-sg-bastion	sg-000df8e5f3e3db3d9	VPC A
ewb-sg-app-a	sg-07f1f640aeb9bf6cc	VPC A
ewb-sg-app-b	sg-029d981d5c0c41c06	VPC B
ewb-bastion (t3.micro)	i-002249c4a53bd78b8	10.10.1.10 + public IPv4 3.107.199.114
ewb-app-a (t3.micro)	i-09ebe0554c6ea71d2	10.10.2.10 / no public IP
ewb-app-b (t3.micro)	i-0cf8eaf3d0fc793f8	10.20.1.10 / no public IP

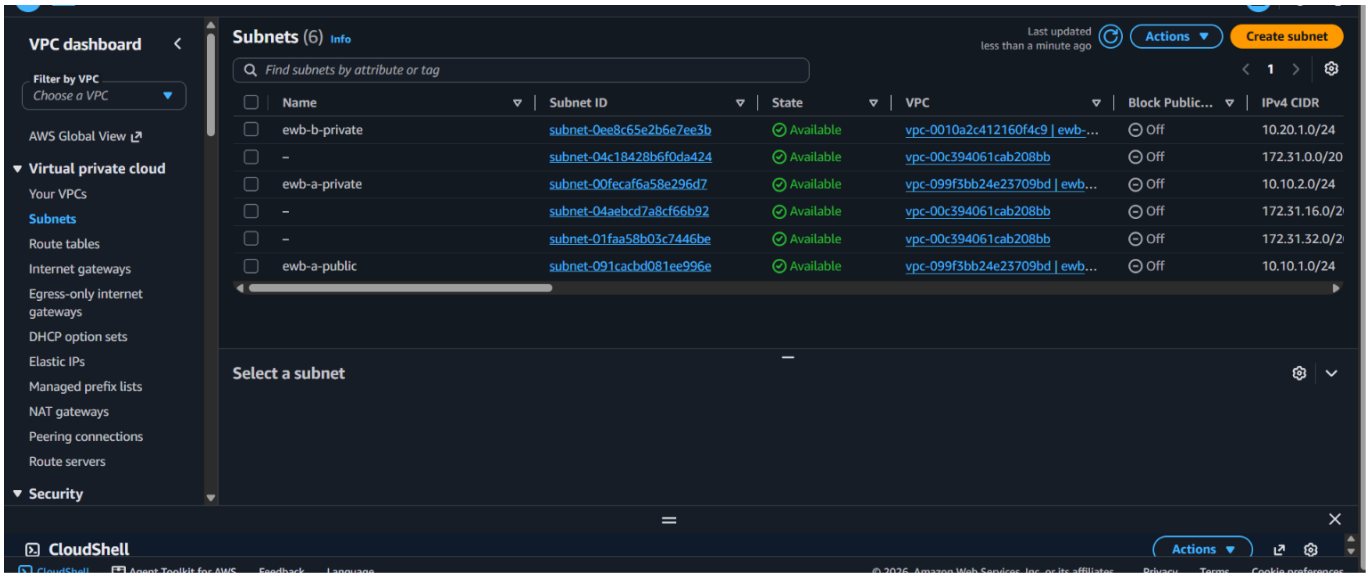
All resources carry the tags Project = EWB-VPC-Peering and Student = OmPanwar.

2. Subnets, internet gateway and route tables

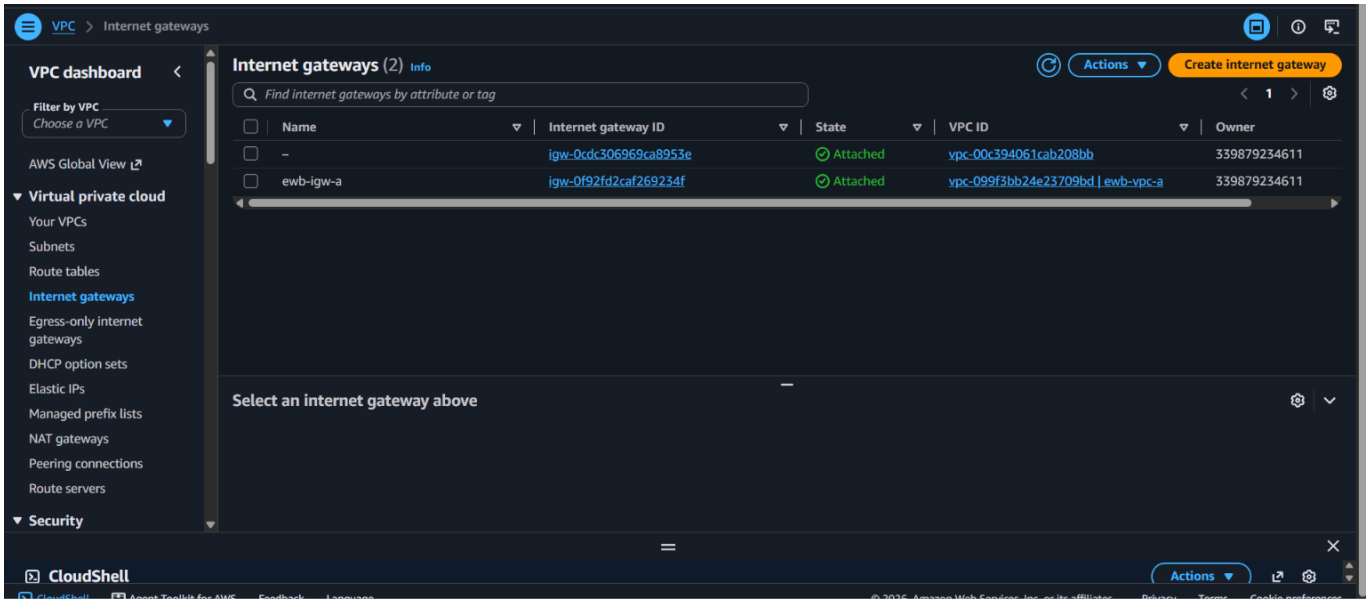
Screenshot 1 - Your VPCs: ewb-vpc-a and ewb-vpc-b (host: AWS console, VPC)



Screenshot 2 - Subnets: ewb-a-public 10.10.1.0/24, ewb-a-private 10.10.2.0/24, ewb-b-private 10.20.1.0/24



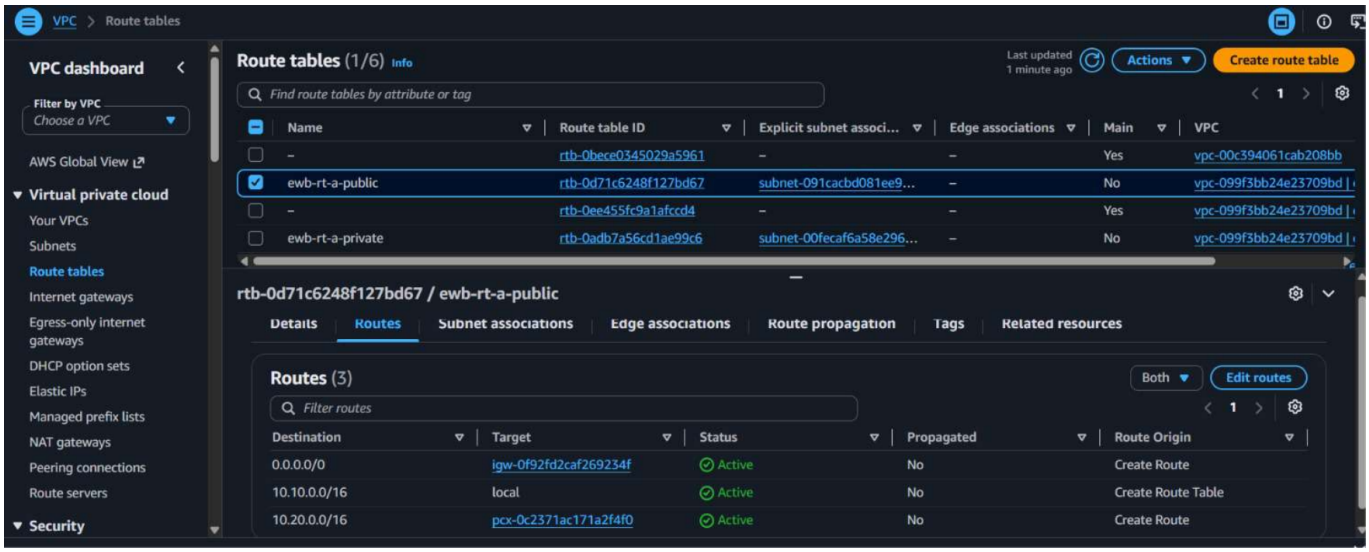
Screenshot 3 - Internet gateway ewb-igw-a attached to ewb-vpc-a (VPC B has none)



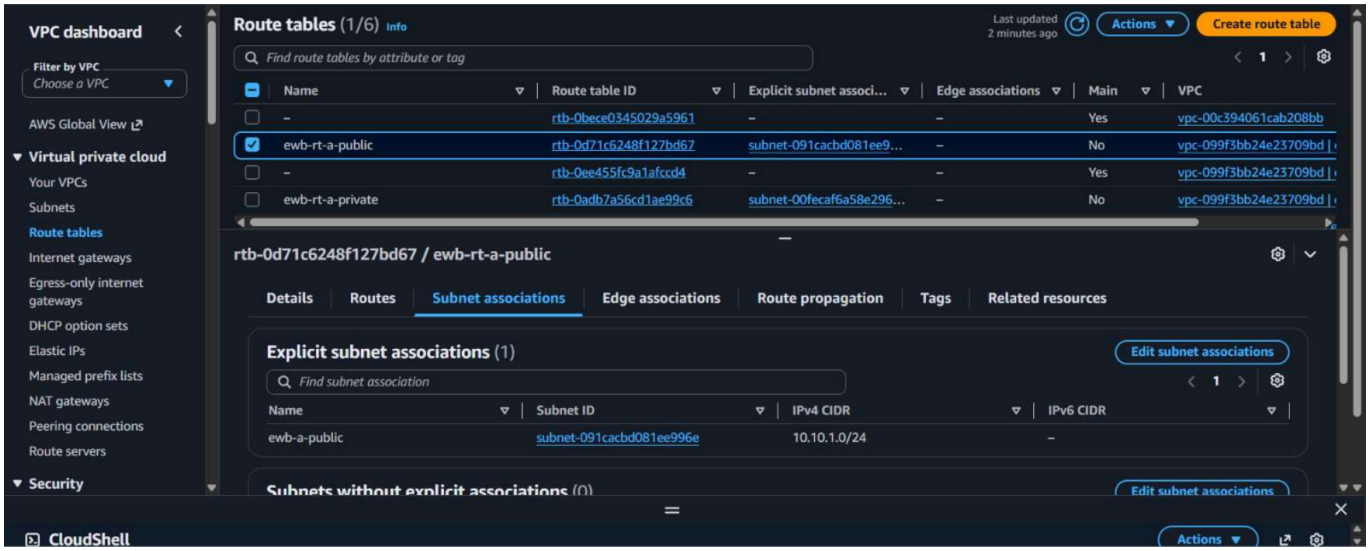
Route tables and subnet associations

Each route table is explicitly associated with its subnet. The peering routes were added on both sides, and the public route table also has the 10.20.0.0/16 route so the bastion can open its ProxyJump SSH connection to App-B.

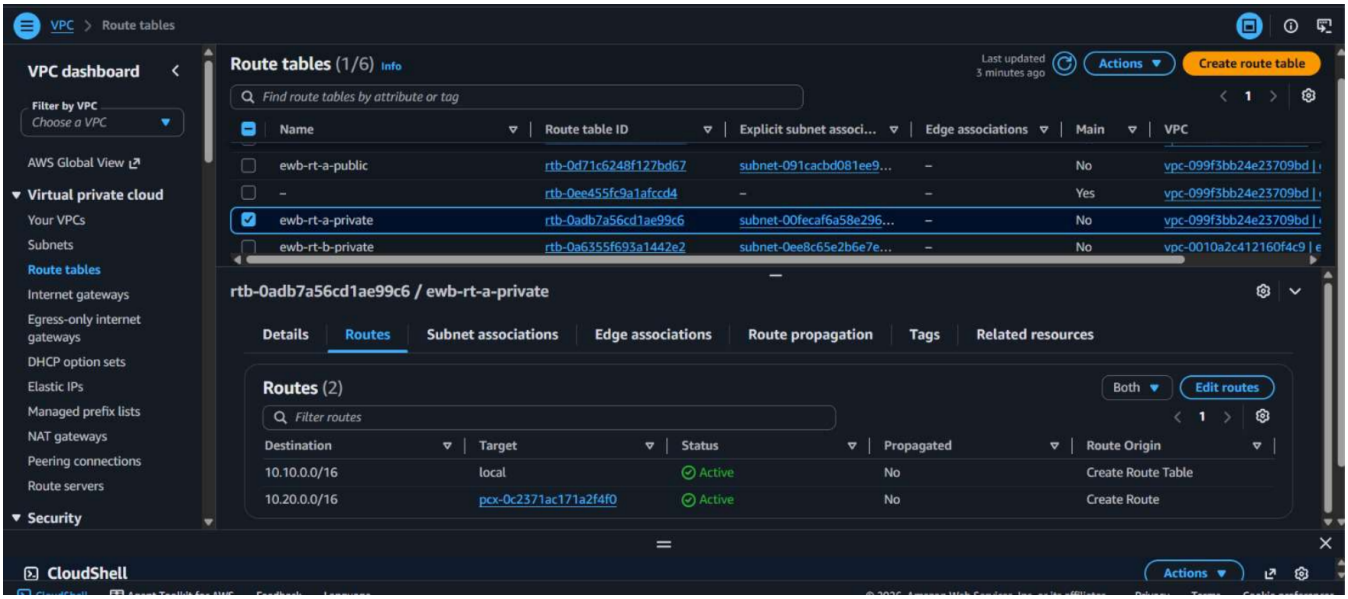
Screenshot 4 - ewb-rt-a-public / Routes: 0.0.0.0/0 to igw, 10.10.0.0/16 local, 10.20.0.0/16 to pcx



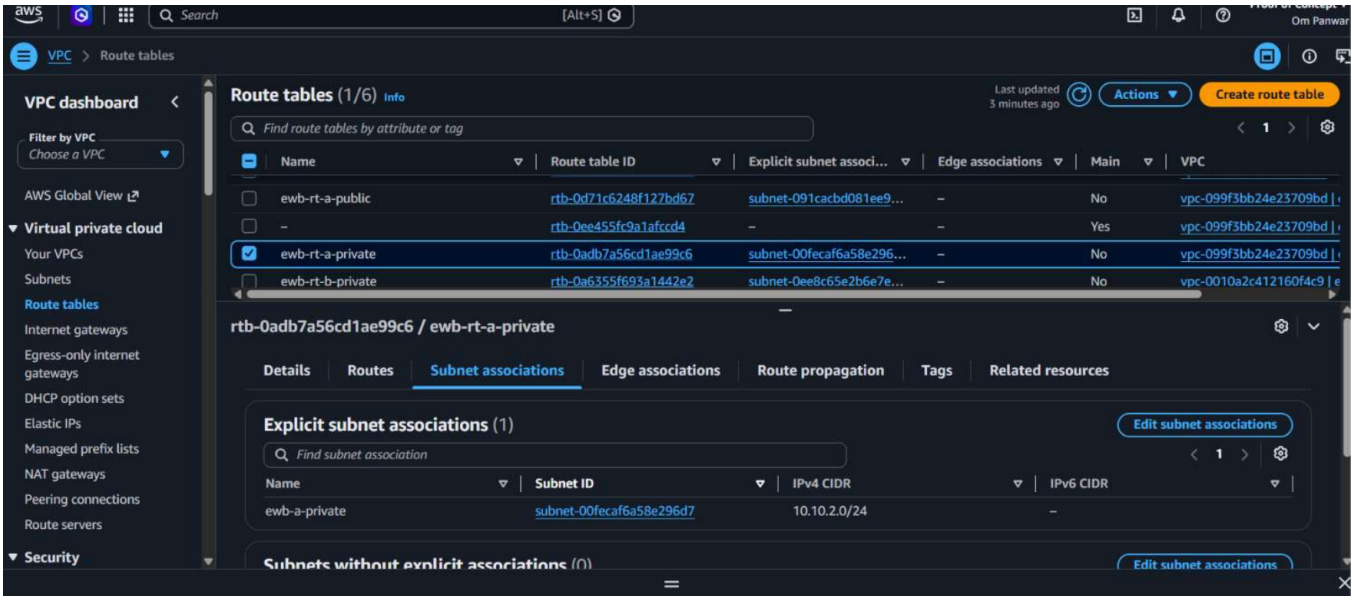
Screenshot 5 - ewb-rt-a-public / Subnet associations: ewb-a-public (10.10.1.0/24)



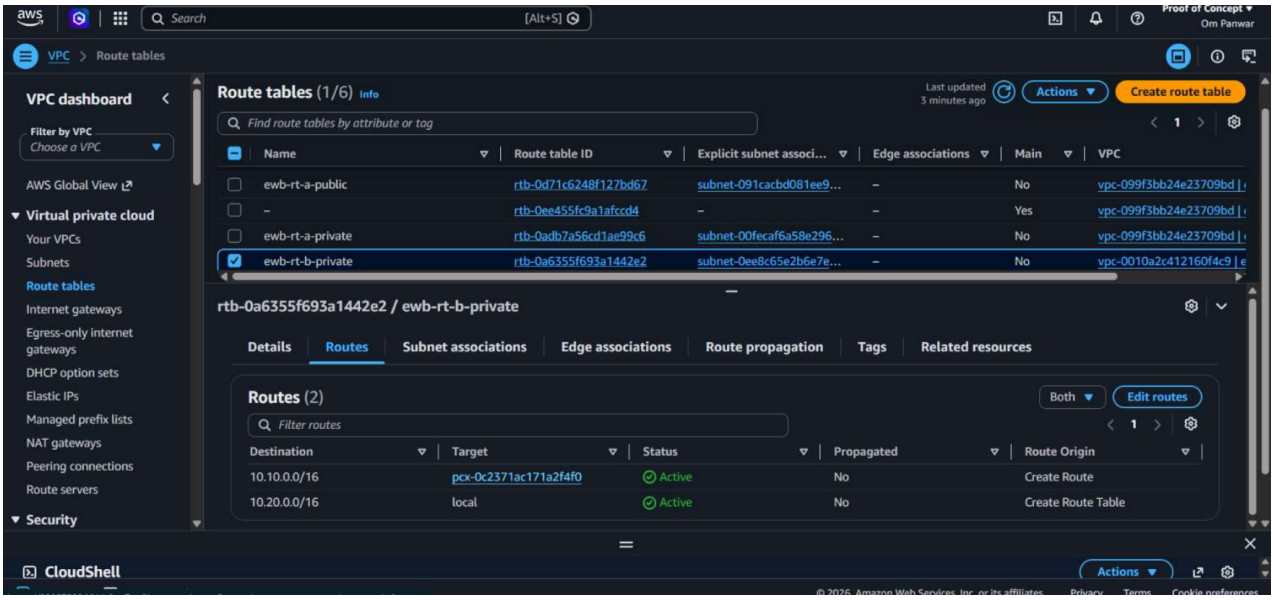
Screenshot 6 - ewb-rt-a-private / Routes: 10.10.0.0/16 local, 10.20.0.0/16 to pcx (no internet route)



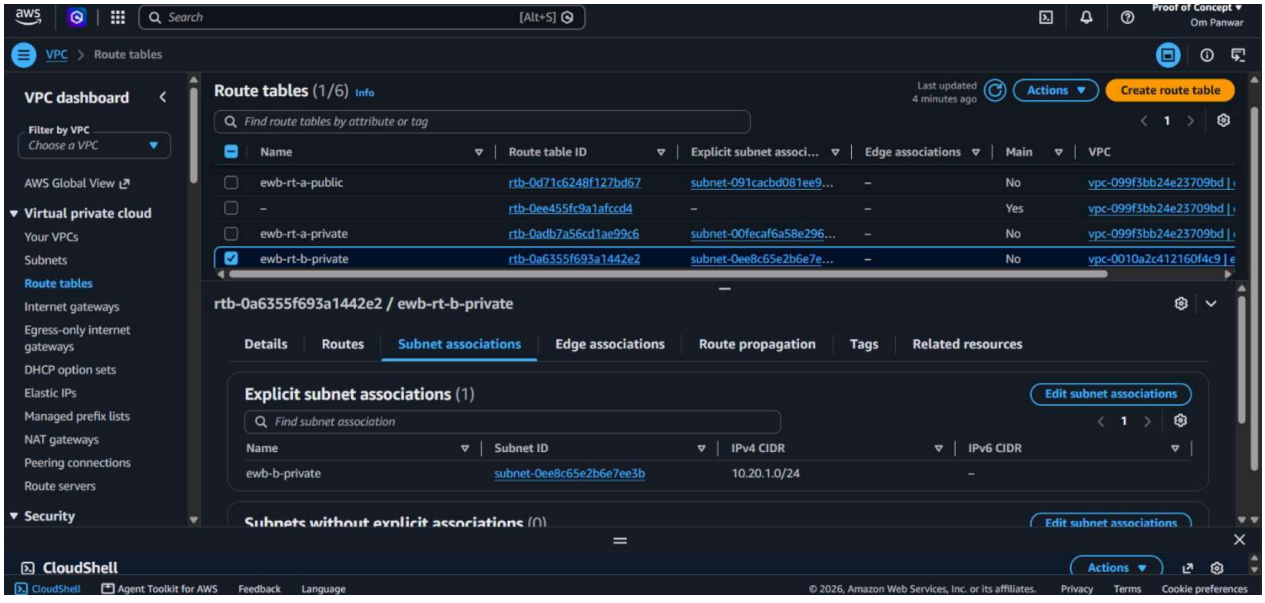
Screenshot 7 - ewb-rt-a-private / Subnet associations: ewb-a-private (10.10.2.0/24)



Screenshot 8 - ewb-rt-b-private / Routes: 10.10.0.0/16 to pcx (return path), 10.20.0.0/16 local

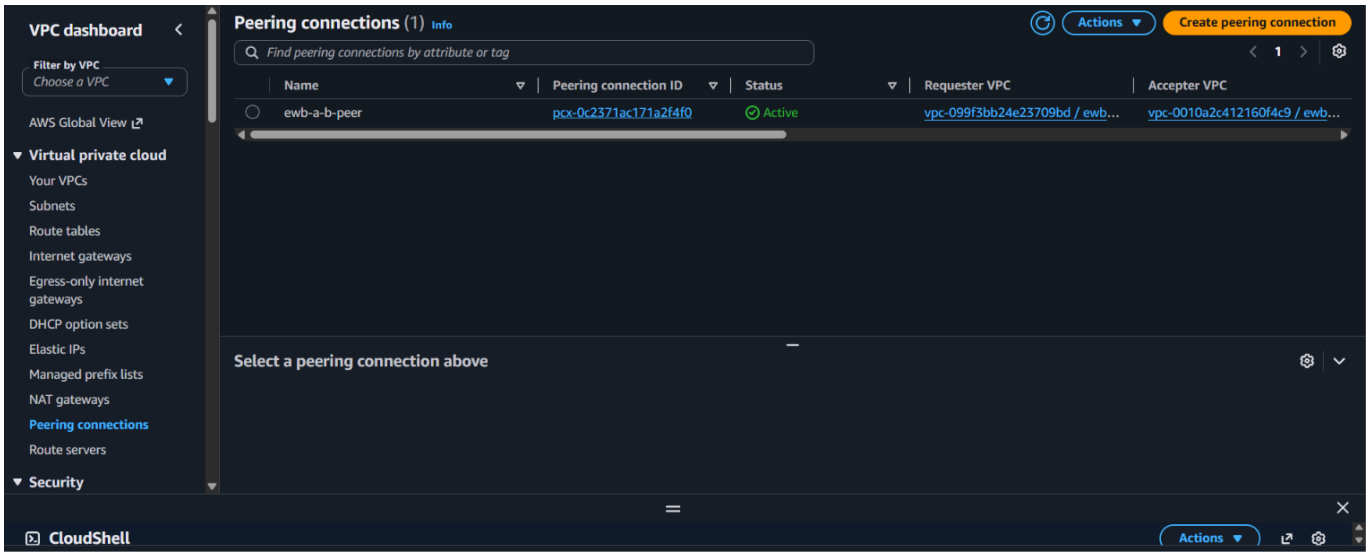


Screenshot 9 - ewb-rt-b-private / Subnet associations: ewb-b-private (10.20.1.0/24)



3. Peering connection and security group rules

Screenshot 10 - Peering connection ewb-a-b-peer (pcx-0c2371ac171a2f4f0): status Active, requester VPC A, accepter VPC B

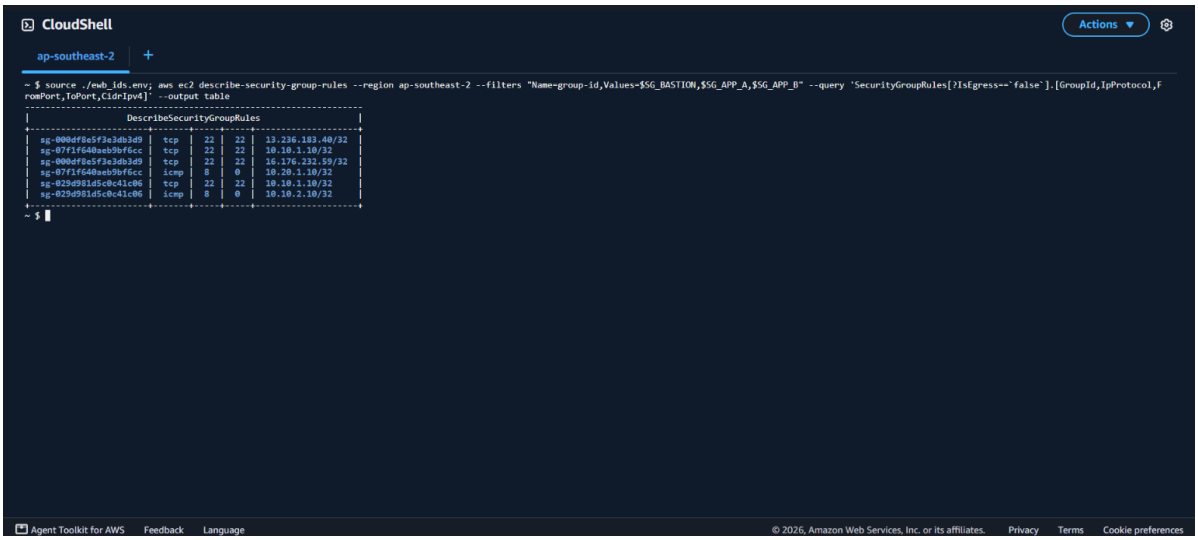


Inbound security group rules

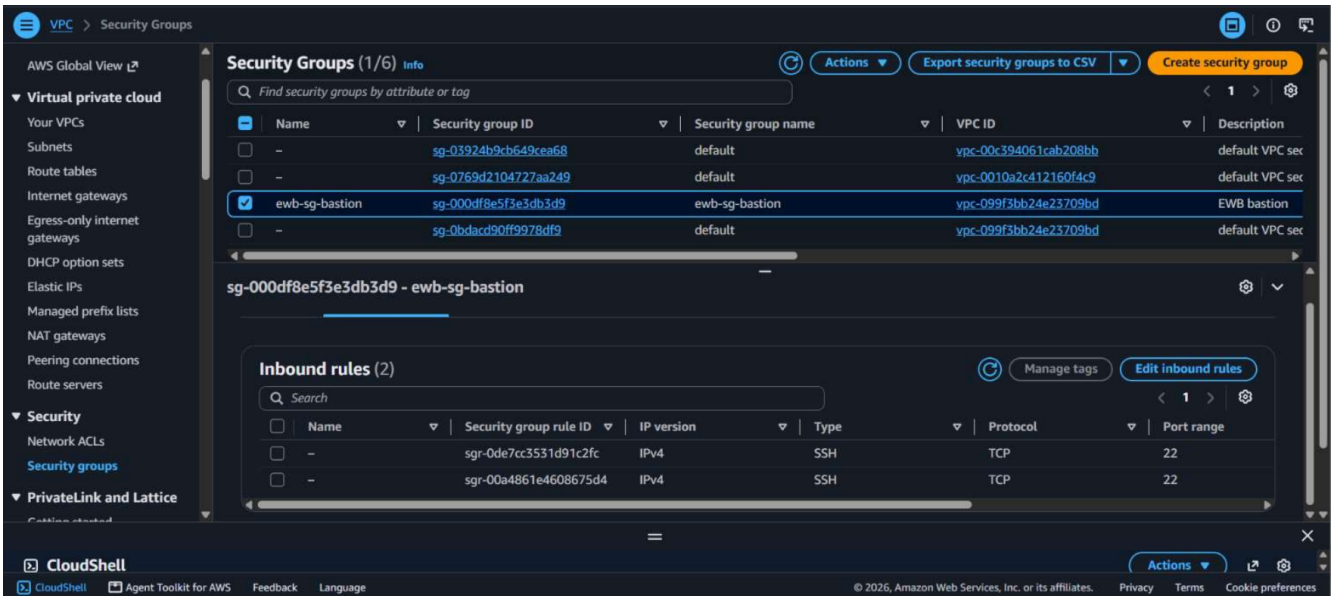
Security group	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	13.236.183.40/32 (my CloudShell public IP, first session)
ewb-sg-bastion (VPC A)	SSH / TCP 22	16.176.232.59/32 (my CloudShell public IP, later session)
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	ICMP echo request (type 8)	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	ICMP echo request (type 8)	10.10.2.10/32 (App-A)

Note: the bastion has two SSH sources because my CloudShell public IP changed between sessions; the new /32 was added for the later session. Nothing is open to 0.0.0.0/0.

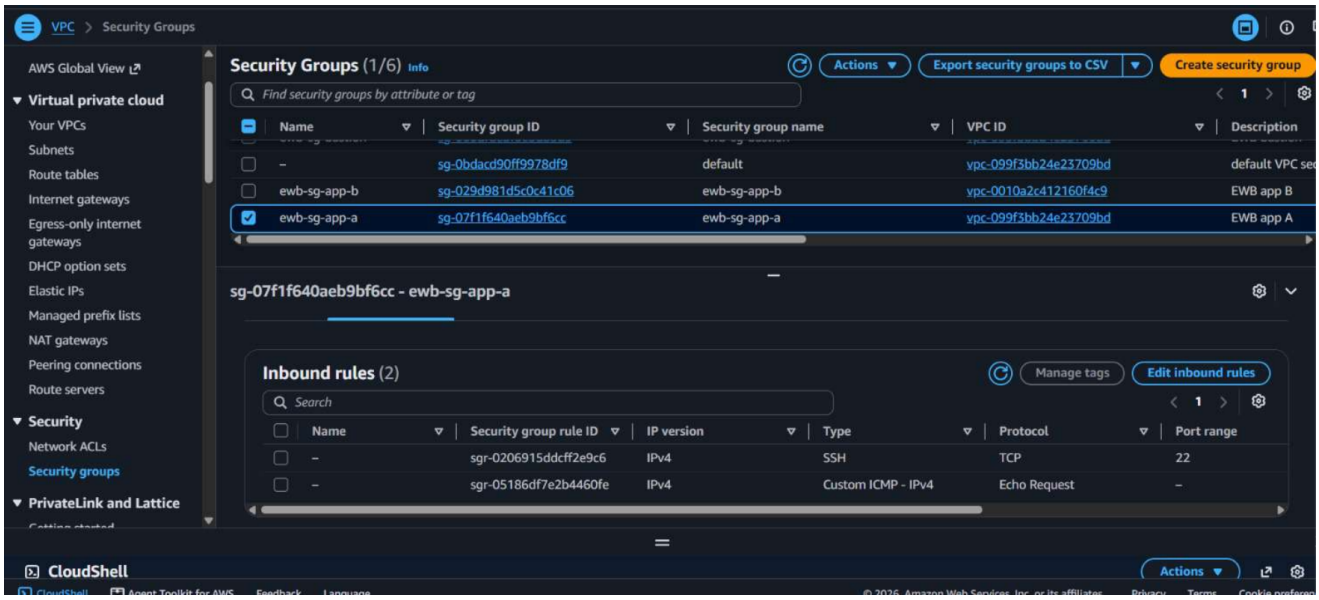
Screenshot 11 - All inbound rules with sources (CLI output, host: CloudShell). sg-000df8e5... = bastion, sg-07f1f640... = app-a, sg-029d981d... = app-b



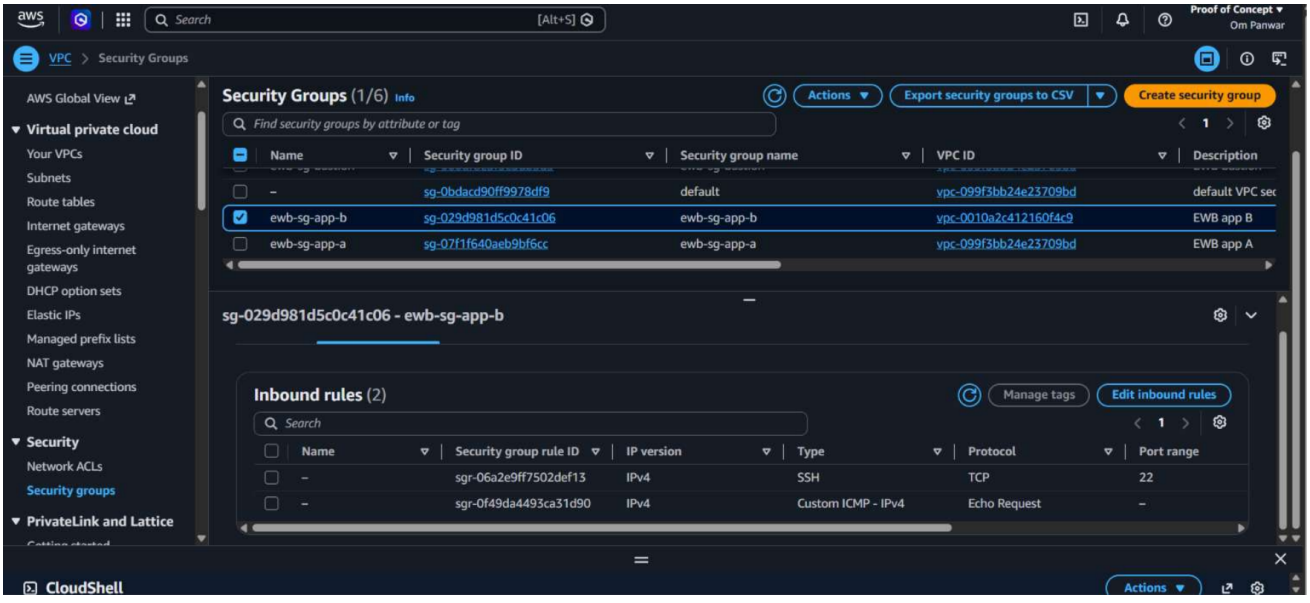
Screenshot 12 - ewb-sg-bastion inbound rules (console)



Screenshot 13 - ewb-sg-app-a inbound rules (console): SSH + ICMP Echo Request

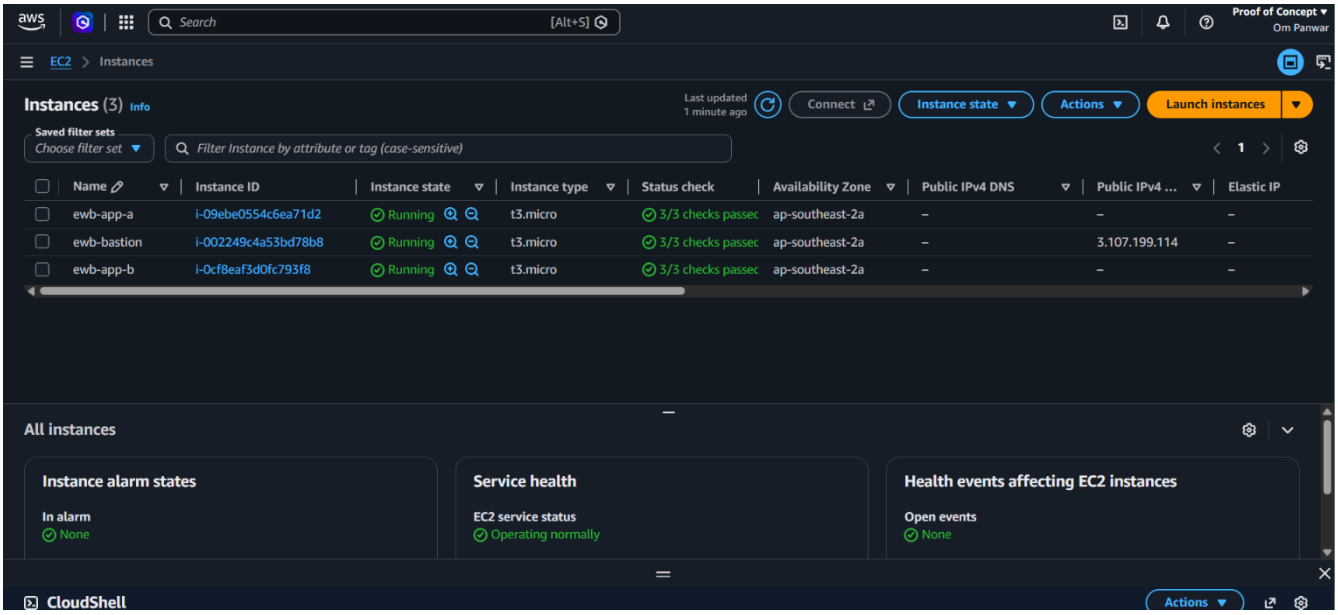


Screenshot 14 - ewb-sg-app-b inbound rules (console): SSH + ICMP Echo Request

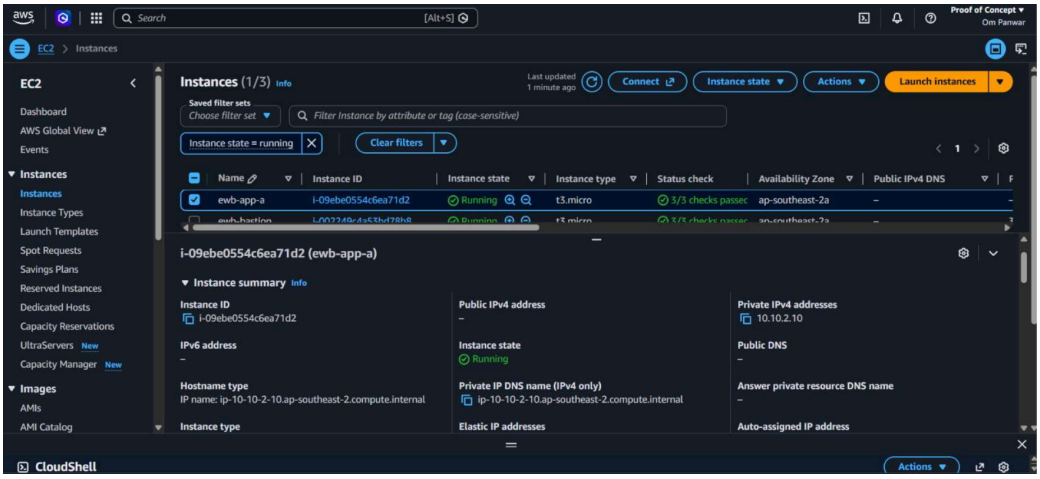


Instances and isolation check

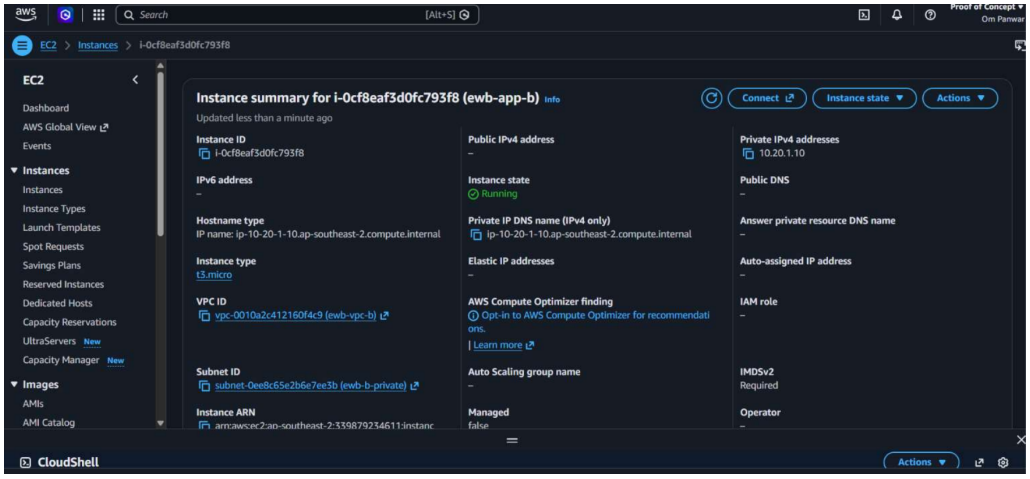
Screenshot 15 - All three instances running (3/3 checks). Only ewb-bastion has a public IPv4 (3.107.199.114)



Screenshot 16 - ewb-app-a (i-09ebe0554c6ea71d2): Public IPv4 address empty, private IP 10.10.2.10



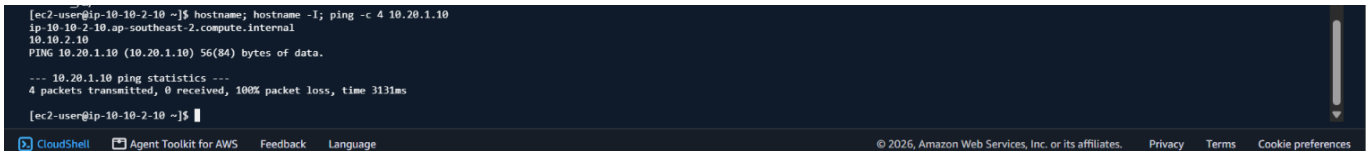
Screenshot 17 - ewb-app-b (i-0cf8eaf3d0fc793f8): Public IPv4 address empty, private IP 10.20.1.10, VPC ewb-vpc-b



4. Test evidence

All tests use private destination IPs. Terminal output below is from my own run (SSH from AWS CloudShell through the bastion with ProxyJump).

Test 01 - Baseline (host: ewb-app-a, BEFORE peering): ping -c 4 10.20.1.10 -> 4 transmitted, 0 received, 100% packet loss



Test 02 - Forward path (host: ewb-app-a, AFTER peering + routes): ping -c 4 10.20.1.10 -> 4 received, 0% packet loss



Test 03 + 04 - Reverse path and private SSH (host: ewb-app-b reached with ssh ewb-app-b through the bastion): ping -c 4 10.10.2.10 -> 4 received, 0% packet loss



5. Controlled failure and recovery

I removed only the ICMP Echo request rule (source 10.10.2.10/32) from ewb-sg-app-b and kept the SSH rule in place. The forward ping then failed. After restoring the exact same rule, the ping succeeded again.

Test 05 - Controlled failure (host: ewb-app-a): ICMP rule removed -> ping -c 4 10.20.1.10 -> 100% packet loss



Test 06 - Recovery (host: ewb-app-a): rule restored -> ping -c 4 10.20.1.10 -> 4 received, 0% packet loss



Explanation: routing and peering were unchanged during this test. Only the destination security group changed, so the packets reached VPC B but App-B's security group dropped the echo requests. This proves that an Active peering connection plus correct routes is not enough on its own; the security group must also allow the traffic.

6. Forward path and return path

Forward path (App-A to App-B)

App-A (10.10.2.10) sends a packet to 10.20.1.10. The route table ewb-rt-a-private matches 10.20.0.0/16 and sends it to the peering connection pcx-0c2371ac171a2f4f0. The peering connection delivers it into VPC B, where ewb-sg-app-b allows ICMP echo request from 10.10.2.10/32.

Return path (App-B to App-A)

App-B (10.20.1.10) replies to 10.10.2.10. The route table ewb-rt-b-private matches 10.10.0.0/16 and sends the reply to the same peering connection, which delivers it back to VPC A. Security groups are stateful, so the reply to an allowed request is permitted automatically.

Administration path

Laptop/CloudShell connects by SSH to the bastion (public subnet, internet gateway route). With ProxyJump, the bastion opens a separate SSH connection to App-A (local route) or App-B (10.20.0.0/16 route via peering in ewb-rt-a-public). App-B therefore sees the bastion's private IP 10.10.1.10 as the source.

7. Viva answers

Question	Answer
1. Why is the return route needed?	Replies need a path back to the sender. Without 10.10.0.0/16 to pcx in VPC B's route table, requests arrive but replies are dropped, so the connection fails. Peering routing is not automatic or symmetric.
2. Why is the App-B SSH source the bastion private IP?	ProxyJump makes the bastion open a new SSH connection to App-B, so App-B sees the bastion's private IP (10.10.1.10), not the laptop's address.
3. Why does Active peering not guarantee a successful ping?	Active only means the link exists. Route tables must point each side's traffic to the pcx, and security groups (and NACLs) must allow the traffic. My controlled failure showed a rule alone can stop it.
4. Can VPC B use VPC A's internet gateway over peering?	No. VPC peering is non-transitive and does not forward traffic to an internet gateway in the peer VPC. VPC B has no internet access in this design.

8. What I learned

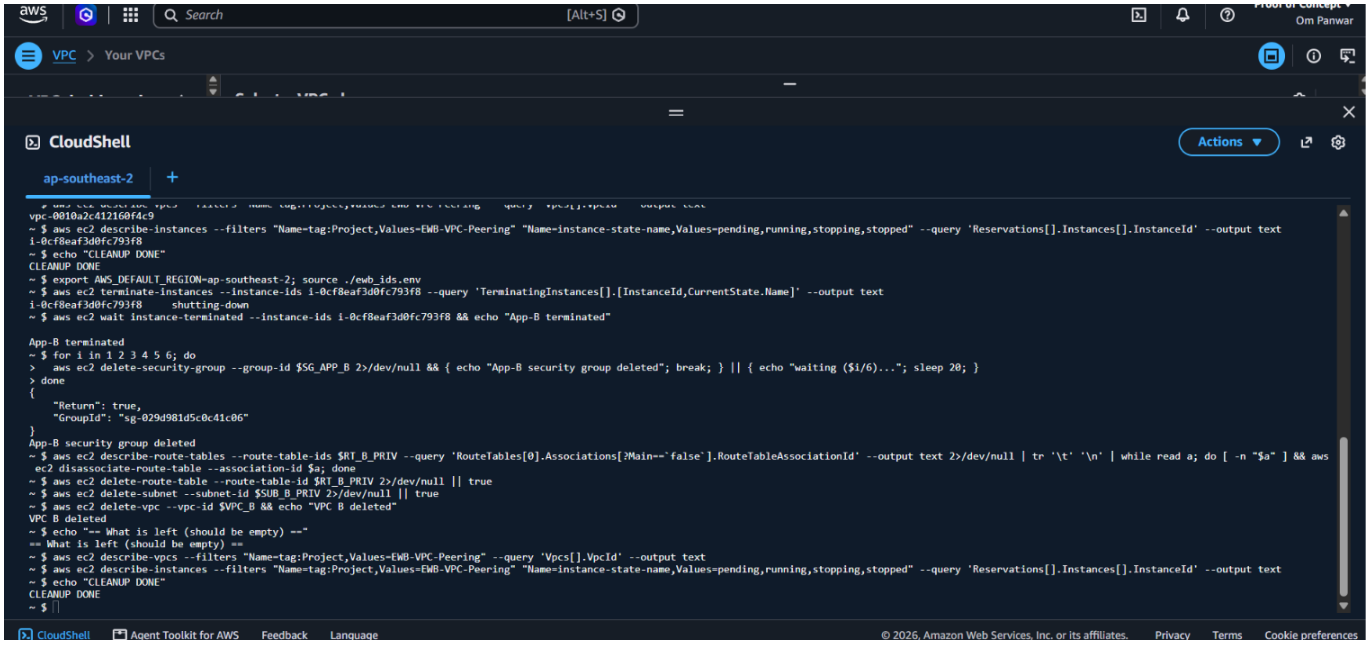
This project showed me that VPC peering is only the link between two networks. For traffic to actually flow I also needed non-overlapping CIDRs, routes in both directions, the correct security group sources and a valid way to administer private hosts. The baseline failure before peering and the controlled failure after it helped me see exactly which layer was responsible: peering and routes decide whether a path exists, and security groups decide whether the traffic is allowed.

I also learned to keep application servers private and use a single bastion with SSH ProxyJump, so the private key never has to be copied to a server, and to scope SSH and ICMP rules to individual /32 addresses instead of 0.0.0.0/0.

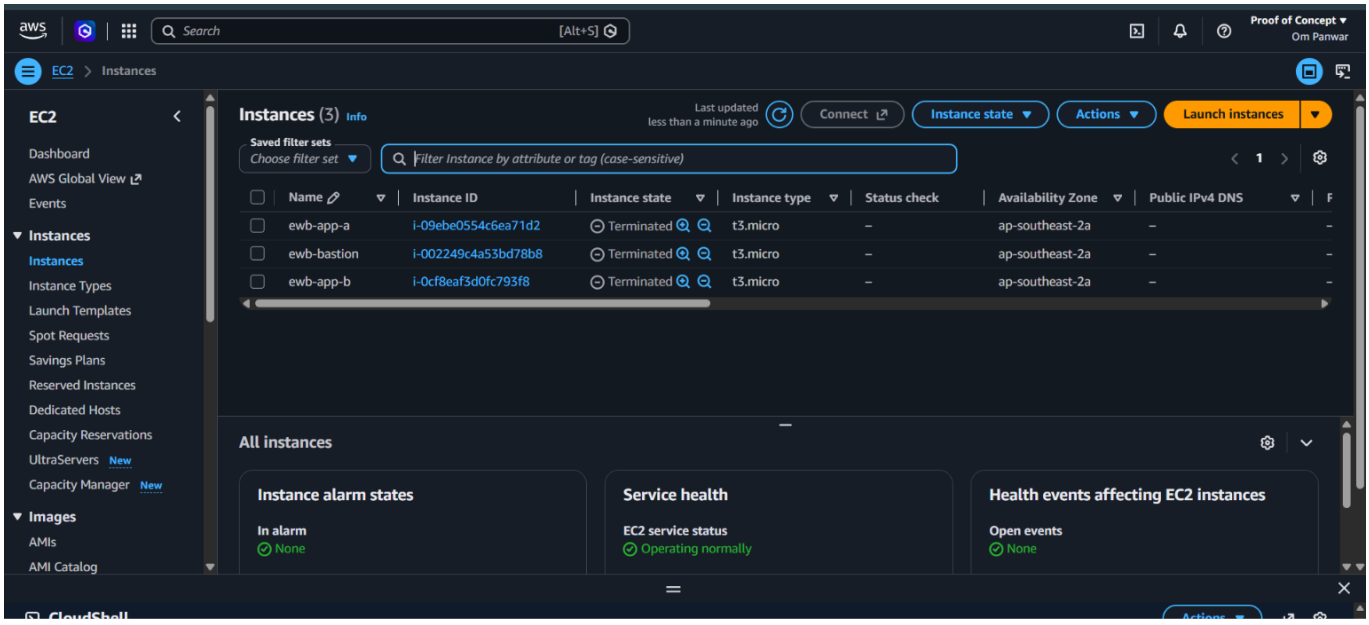
9. Cleanup evidence

All lab resources were deleted after the evidence was captured: instances terminated, peering connection and routes deleted, security groups, subnets, route tables, internet gateway and both VPCs deleted, and the lab key pair removed.

Cleanup 1 - CloudShell: ewb-app-b was terminated by instance ID after the first cleanup pass left it running, then its security group, route table, subnet and ewb-vpc-b were deleted. The 'What is left' check returns nothing and prints CLEANUP DONE.



Cleanup 2 - EC2 Instances: ewb-app-a, ewb-bastion and ewb-app-b are all Terminated



Cleanup 3 - VPC Your VPCs: only the default VPC remains; ewb-vpc-a and ewb-vpc-b are deleted

